

la declaración de los tipos de datos

Fundamentos de la programación 1

Grado en Desarrollo de Videojuegos

Examen de convocatoria extraordinaria. Curso 25/26

Indicaciones generales:

- Se proporciona una **plantilla** con el esquema del programa y algunas declaraciones.
 - Las líneas 1 y 2 serán comentarios de la forma:

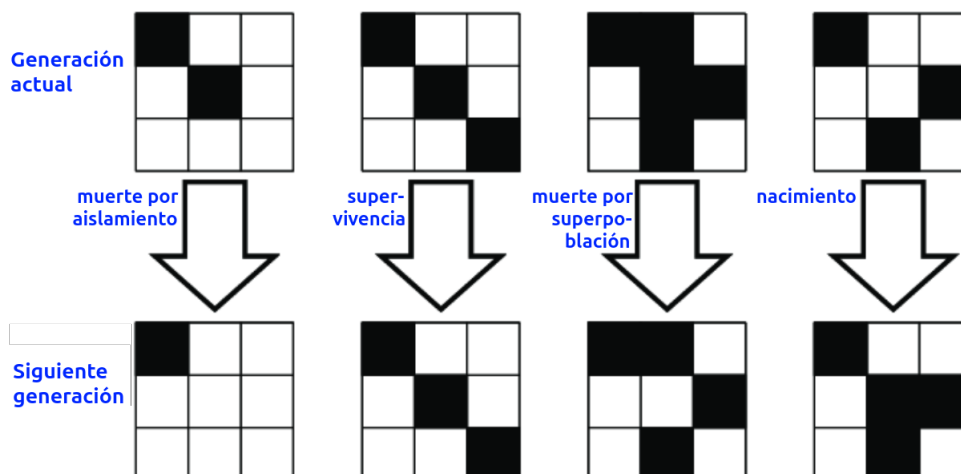
```
// Nombre Apellido1 Apellido2  
// Laboratorio, puesto
```
 - **Lee atentamente el enunciado** e implementa el programa tal como se pide, con los métodos, parámetros y requisitos que se especifican. No puede modificarse la representación propuesta, ni alterar los parámetros de los métodos especificados. No se especifica el modo de paso de los parámetros en los métodos (**out**, **ref**, ...), que debe determinar el alumno.

Pueden implementarse todos los métodos adicionales que se considere oportuno, especificando claramente su cometido, parámetros, etc.
 - **El programa debe compilar y funcionar correctamente, y estar bien estructurado y comentado.** Se valorarán la claridad, la concisión y la eficiencia.
 - La **entrega** se realizará a través del servidor FTP. Se entregará únicamente el archivo `Program.cs` comprimido en un archivo `.zip` con el nombre de la forma `Nombre_Apellido1_Apellido2.zip`.
-

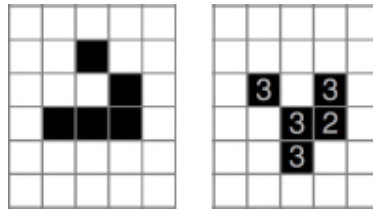
Vamos a implementar el **juego de la vida**, una simulación de vida artificial desarrollada por John Horton Conway en 1970. Consiste en una cuadrícula en la que se desarrolla la vida de una colonia de células, gobernada por tres reglas que determinan la siguiente generación:

- **Supervivencia:** cada célula con 2 ó 3 vecinos sobrevive y pasa a la generación siguiente.
- **Fallecimiento:** cada célula con 4 o más vecinos muere por super-población y cada célula con menos de 2 vecinos muere por aislamiento.
- **Nacimiento:** en cada casilla vacía con exactamente 3 células vecinas nace una nueva célula.

A continuación se ilustra la aplicación de estas reglas a la casilla central de cada imagen de acuerdo al contenido de sus casillas vecinas (las células se representan con casillas negras):

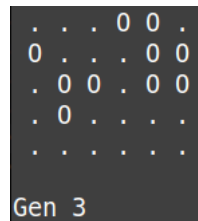


Estas reglas se **aplican simultáneamente a cada casilla** de una generación para producir la siguiente. Las casillas de la primera y última filas son vecinas, al igual que las de la primera y última columnas, con lo que tendremos una superficie toroidal o "donut". Así, todas las casillas tienen exactamente 8 posiciones vecinas (las de alrededor, excluida ella misma). A continuación se muestra una población y la generación siguiente que resulta de aplicar las reglas:



Representaremos el estado mediante una matriz de booleanos de tamaño `fils` $\times$ `cols`, donde `true` representa célula y `false` casilla vacía. Se pide implementar los siguientes métodos:

- [0.5] `void Inicializa(int fils, int cols, bool [,] mat)`: crea la matriz booleanos `mat` de tamaño `fils` $\times$ `cols` con las componentes rellenas aleatoriamente con célula o casilla vacía, con probabilidad 0.5 en ambos casos. Recordemos que `rnd.Next(a,b)` devuelve un número entero aleatorio del intervalo `[a,b-1)`.
- [0.5] `void Render(bool [,] mat, int gen)`: dibuja en pantalla el estado dado en la matriz `mat` utilizando '0' para las células y '.' para las casillas vacías. Para proporcionar el aspecto, cada casilla ocupará **dos espacios horizontales**. Además indicará el número de generación actual `gen`. El aspecto será el siguiente:



Para calcular el número de vecinas en la superficie toroidal serán útiles los siguientes métodos, que obtienen la fila o columna previa o siguiente a una dada, con comportamiento circular.

- [0.5] `int Prev(int n, int tam)`: devuelve el número anterior a `n` en un intervalo de tamaño `tam` con comportamiento circular. Por ejemplo, si `n=0` y `tam=5`, el resultado será 4; si `n=3` y `tam=5`, el resultado será 2.

Análogamente, `int Next(int n, int tam)` devuelve el siguiente. Por ejemplo, si `n=0` y `tam=5`, el resultado será 1; si `n=4` y `tam=5`, el resultado será 0.

- [1] `int NumVecinas(bool [,] mat, int fil, int col)`: devuelve el número de células que hay en las 8 posiciones vecinas a la casilla de coordenadas (`fil`, `col`) en la matriz `mat`. Los métodos `Prev` y `Next` serán útiles para acceder a las casillas vecinas, incluso en los bordes de la matriz.
- [1] `bool [,] SiguienteGen(bool [,] mat, int gen)`: dada población de células representada en `mat`, calcula la generación siguiente aplicando las reglas expuestas al principio del enunciado, y la devuelve EN una nueva matriz (del mismo tamaño). Para ello utilizará el método anterior `NumVecinas`. Nótese que cada casilla en la nueva población depende únicamente de los vecinos de la población anterior y no de los nacimientos o muertes que se produzcan en la nueva población. Este método además incrementa en 1 el número de generación `gen`.

Con estos métodos ya puede implementarse una primera versión de **Main** que genere una población inicial y, en el bucle principal, vaya mostrando sucesivas generaciones, poniendo un retardo entre ellas para poder apreciar los cambios (con el método `Thread.Sleep(500)`). A continuación, se pide implementar los siguientes métodos:

- `[1] bool Estable(bool [,] mat1, bool [,] mat2)`: determina si la población ha alcanzado una configuración estable, es decir, una población que ya no cambia en sucesivas generaciones. Para ello considera dos poblaciones consecutivas `mat1` y `mat2`, y comprueba si son iguales (por ejemplo, una población con 4 células formando un cuadrado es estable).
- `[1] void LeeArchivo(string file, bool [,] mat)`: lee del archivo `file` la configuración de una colonia de células y crea la matriz `mat` con esa configuración. El archivo tendrá dos líneas iniciales con el número de filas y columnas, y a continuación la matriz propiamente dicha, con '0' para las casillas vacías y '.' para las células. Por ejemplo, para la población mostrada en el método `Render`, el archivo sería:

```
5
6
...00.
0...00
.00.00
.0....
.....
```

- `[1] void Menu(bool [,] mat)`: pregunta al usuario si quiere generar una población aleatoria inicial o leerla de un archivo. En el primer caso solicitará las dimensiones `files × cols` y utilizará el método `Inicializa` para crear la matriz. En el segundo, solicitará el nombre del archivo y utilizará el método `LeeArchivo`.

Esta simulación tiene una gran variedad de patrones (grupos de células) interesantes. Por ejemplo, 4 células formando un cuadrado configuran un patrón estable, que no cambia entre generaciones. A continuación buscaremos patrones en la población, que en definitiva es **buscar submatrices** dentro de la matriz:

- `[1] bool BuscaPatronPos(bool [,] mat, bool [,] pat, int fil, int col)`: determina si la matriz `pat` se encuentra en `mat` con su esquina superior izquierda en las coordenadas `(fil, col)`, teniendo en cuenta que la superficie es toroidal.
- `[1] void BuscaPatron(bool [,] mat, bool [,] pat, out int fil, out int col)`: busca la submatriz `pat` en `mat` y devuelve las coordenadas `(fil, col)` de la esquina superior izquierda donde se ha encontrado; devuelve `(-1, -1)` si no se encuentra.

La versión definitiva de **Main** quedará de la siguiente forma:

- **[1.5] Main**: llama a `Menu` para obtener la población inicial y luego implementará el bucle principal de la simulación. Este bucle obtiene y dibuja sucesivas generaciones de la población inicial hasta obtener una configuración estable (método `Estable`).

Además, en cada generación comprobará si se encuentra el patrón dado en la plantilla (en la variable `patron`); en ese caso, pausará el bucle (con `Console.ReadLine()`), mostrará un mensaje indicando la posición donde se ha encontrado y esperará a que el usuario pulse `intro` para continuar la simulación.